

第17章: イベント機能

本章では、盆栽展示会や講習会などのイベント情報を管理・表示する機能を実装します。カレンダー表示、地域フィルタ、過去イベントの扱い、URLパラメータによる状態管理、外部サイトからのスクレイピング、そしてCronジョブによる自動クリーンアップを学びます。

この章の全体像

盆栽の世界では、展示会、即売会、講習会などさまざまなイベントが各地で開催されています。本章では、こうしたイベント情報を「登録」「検索」「カレンダー表示」「自動管理」できる 本格的なイベント管理システムを構築します。

身近な例で言えば、「地域のお祭り情報サイト」や「コンサートチケットサイト」の イベント一覧機能に近いものを作ると考えてください。

17.0 実習手順の進め方と手順マップ

手順に沿って進めると、**どのファイルに何を入力し、何を確認すればよいか** が分かります。形式の説明は [チュートリアルの進め方](#) を参照してください。

手順	主な対象ファイル (例)	完了時に確認すること
イベントモデル	<code>prisma/schema.prisma</code>	Event が定義され CRUD が動く
イベント CRUD	<code>lib/actions/event.ts</code> 等	作成・一覧・編集・削除ができる
カレンダー表示	<code>components/event/EventCalendar*.tsx</code>	カレンダーでイベントが表示される
地域フィルタ・過去表示	フィルターコンポーネント	地域・過去イベントの切替が動く
Cron・スクレイピング	<code>app/api/cron/*</code> , スクレイピング	自動クリーンアップ・インポートが動く

各セクションで **対象ファイル・入力するコード（サンプルコード）・実行方法・実行するとうなる・このあと変わること・確認方法**を確認しながら進めてください。

この章で必要な前提知識

知識	説明	参照章
Next.js App Router	ページ作成とルーティングの基本	第2章
Server Components	サーバー側でデータを取得する仕組み	第3章
Server Actions	フォーム送信によるデータ変更	第5章
Prisma	データベース操作の基本	第4章
URLSearchParams	ブラウザのURL操作	JavaScript基礎

この章のファイル構成

```
app/(main)/events/
├─ page.tsx          # イベント一覧ページ
├─ loading.tsx       # ローディングスケルトン
├─ error.tsx         # エラーバウンダリ
├─ new/
│   └─ page.tsx      # 新規登録ページ
└─ [id]/
    ├─ page.tsx      # イベント詳細ページ
    ├─ loading.tsx   # 詳細ローディング
    ├─ error.tsx     # 詳細エラー
    ├─ DeleteEventButton.tsx # 削除ボタン（ローカル）
    └─ edit/
        └─ page.tsx  # 編集ページ

components/event/
├─ EventCard.tsx     # イベントカード
├─ EventList.tsx     # イベント一覧グリッド
├─ EventCalendar.tsx # カレンダー本体
├─ EventCalendarWrapper.tsx # カレンダーSSR無効化ラッパー
├─ EventForm.tsx     # 作成・編集フォーム
├─ RegionFilter.tsx  # 地域フィルター
├─ ShowPastToggle.tsx # 過去イベント表示切替
├─ EventFilterPersistence.tsx # フィルター永続化
├─ DeleteEventButton.tsx # 削除ボタン（共通）
└─ EventActionsDropdown.tsx # アクションメニュー

lib/actions/
├─ event.ts          # イベントCRUD Server Actions
└─ event-import.ts   # スクレイピング・インポート

lib/scraping/
└─ bonsai-events.ts  # 外部サイトスクレイピング

app/api/cron/
└─ cleanup-events/
    └─ route.ts      # 古いイベント自動削除Cron
```

17.1 イベント機能の設計

このセクションで学ぶこと

- イベント機能に必要な要件を整理する方法
- 機能を「管理」「表示」「連携」「運用」の4つの観点で分類する考え方
- 実装前に設計を固めることの重要性

機能要件

イベント機能を設計するにあたり、まず「誰が」「何を」「どのように」使うかを整理します。これはソフトウェア開発における「要件定義」と呼ばれるステップです。

レストランのメニューを考えるとときに「どんな料理を出すか（種類）」「どう提供するか（盛り付け）」「お客さんがどう注文するか（インターフェース）」を考えるのと同じです。

1. イベント管理（データの入力と保存）

- タイトル、開催日時、場所、説明
- 都道府県・地方ブロック分類
- 主催者情報、参加費、即売有無
- 公式サイトURL

2. 表示機能（データの見せ方）

- カレンダービュー（月表示） -- 月間の予定を一覧できる
- リストビュー（時系列） -- 直近のイベントを上から順に表示
- 地域フィルタ（都道府県/地方ブロック） -- 自分の地域のイベントだけ絞り込み
- 過去イベントの表示/非表示切替 -- 終了済みイベントを隠す/見る

3. URLパラメータ連携（ユーザー体験の向上）

- フィルタ状態をURLに反映 -- 例: `/events?region=関東&showPast=true`
- ブックマーク・共有可能 -- URLをコピーすれば同じ検索結果を再現
- ブラウザの戻る/進むに対応 -- フィルタ操作も履歴に記録

4. 管理機能（運用と自動化）

- イベントのスクレイピング/インポート -- 外部サイトからデータ取り込み

- 重複チェック -- 同じイベントの二重登録を防止
- 自動クリーンアップ -- 終了後6ヶ月経過したイベントを自動削除

URLクエリパラメータとは？ URLの `?` 以降に付く `キー=値` のペアです：

```
/events?region=関東&showPast=true
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
クエリパラメータ（2つ）
```

Next.jsでは `searchParams` として取得できます：

```
// Next.js 15+では searchParams は Promise
export default async function Page({ searchParams }: {
  searchParams: Promise<{ region?: string; showPast?: string }>
}) {
  const params = await searchParams
  const region = params.region      // '関東'
  const showPast = params.showPast // 'true' (文字列)
}
```

機能の全体像

```
[ユーザー（一般）] → [イベント閲覧：カレンダー/リスト/地域フィルタ] → [PostgreSQL eventsテーブル]
[管理者]           → [イベント登録：手動入力/スクレイピング]       → [PostgreSQL eventsテーブル]
[Cron（自動）]     → [自動クリーンアップ：6ヶ月経過分を削除]      → [PostgreSQL eventsテーブル]
```

実装しないとどうなる？

- 設計なしにコードを書き始めると、途中で「あの機能も必要だった」と手戻りが発生します
- URLパラメータ連携がないと、ユーザーが絞り込んだ結果をブックマークできません
- 自動クリーンアップがないと、古いイベントが溜まり続けてパフォーマンスが低下します

▶ 理解度チェック: イベント機能の設計

17.2 データモデル

このセクションで学ぶこと

- イベント情報をデータベースでどのように表現するか
- Prismaスキーマの各フィールドの役割と設計意図
- インデックスを使ったデータベースの検索高速化
- `@map` を使ったフィールド名とカラム名の対応付け

データモデルの考え方

データモデルとは、「どんな情報を、どのような形で保存するか」を定義したものです。現実世界のイベントチラシを思い浮かべてください。チラシには以下の情報が載っています。

- イベント名（タイトル）
- いつ（開始日時・終了日時）
- どこで（会場名・住所・都道府県）
- 何をする（詳細説明）
- 誰が主催（主催者）
- いくら（参加費）
- 詳細はこちら（公式サイトURL）

これらの情報をデータベースのカラム（列）として定義するのがデータモデルです。

Event テーブルの構造

ファイル: `prisma/schema.prisma`

カラム名	型	説明
id	String	一意な識別子 (cuid)
title	String	イベント名 (最大100文字)
description	String?	詳細説明 (省略可能)
start_date	DateTime	開始日時
end_date	DateTime?	終了日時 (省略可能)
prefecture	String?	都道府県 (省略可能)
city	String?	市区町村 (省略可能)
venue	String?	会場名 (省略可能)
organizer	String?	主催者 (省略可能)
admission_fee	String?	入場料 (省略可能)
has_sales	Boolean	即売あり/なし (デフォルト: false)
external_url	String?	外部リンクURL (省略可能)
is_hidden	Boolean	非表示フラグ (デフォルト: false)
hidden_at	DateTime?	非表示にした日時
created_by	String	登録者のユーザーID (必須)
created_at	DateTime	レコード作成日時

インデックス:

- `(start_date)` -- 開始日での検索の高速化
- `(prefecture)` -- 都道府県フィルタの高速化
- `(is_hidden)` -- 非表示フィルタの高速化

Prisma スキーマ定義

```
// prisma/schema.prisma

model Event {
  id          String    @id @default(cuid())
  title       String    @db.VarChar(100)
  description  String?   @db.Text
  startDate   DateTime  @map("start_date")
  endDate     DateTime? @map("end_date")
  prefecture  String?
  city        String?
  venue       String?
  organizer   String?
  admissionFee String?   @map("admission_fee")
  hasSales    Boolean   @default(false) @map("has_sales")
  externalUrl  String?   @map("external_url")
  isHidden    Boolean   @default(false) @map("is_hidden")
  hiddenAt    DateTime? @map("hidden_at")
  createdBy   String    @map("created_by")
  createdAt   DateTime  @default(now()) @map("created_at")

  creator User @relation(fields: [createdBy], references: [id], onDelete: Cascade)

  @@index([startDate])
  @@index([prefecture])
  @@index([isHidden])
  @@map("events")
}
```

期待される結果:

`npx prisma db push` を実行すると、PostgreSQLに以下のようなテーブルが作成されます。

テーブル "events" が正常に作成されました

- 16カラム (id, title, description, start_date, ...)
- 3インデックス (start_date, prefecture, is_hidden)
- 1外部キー (created_by → users.id)

各フィールドの設計ポイント:

- `@db.VarChar(100)`: タイトルの最大文字数をデータベースレベルで制限
- `admissionFee` がString型な理由: 「無料」「大人1,000円/子供500円」のように自由形式で入力できるため
- `isHidden` / `hiddenAt`: 管理者がイベントを非表示にした際の記録用

- `onDelete: Cascade`: ユーザーが退会したら、そのユーザーが作成したイベントも一緒に削除

実装しないとどうなる？

- インデックスがないと、イベント数が増えた際に検索が極端に遅くなります（数百件で数秒かかることも）
- `isHidden` フラグがないと、不適切なイベントを非表示にする手段がなく、削除しか選べません
- `@map` を使わないと、TypeScriptとSQLで命名規則が統一できず、混乱の原因になります

▶ 理解度チェック: データモデル

17.3 地方ブロック・都道府県データ

このセクションで学ぶこと

- TypeScriptの `as const` アサーションによる型安全な定数定義
- 地方ブロックと都道府県の階層構造をコードで表現する方法
- ユーティリティ関数（地方から都道府県リストを取得、都道府県から地方を逆引き）の実装

なぜ定数ファイルとして定義するのか？

地方ブロックや都道府県の一覧は、アプリケーション全体のさまざまな場所で使われます。フォームのドロップダウン、フィルタ機能、データのバリデーションなどです。

もしこれらの値をコンポーネントごとにベタ書きすると、以下の問題が起きます。

- 値を変更するとき、すべての箇所を修正する必要がある
- タイプミスによるバグが発生しやすい
- コードの重複が増える

定数ファイルとして1箇所にまとめることで、これらの問題を防げます。

定数定義の実装

ファイル: `lib/prefectures.ts`

```
// lib/prefectures.ts

/**
 * 都道府県一覧（北から南の順）
 * as const をつけることで値が変更不可能な「リテラル型」になる
 */
export const PREFECTURES = [
  '北海道',
  '青森県', '岩手県', '宮城県', '秋田県', '山形県', '福島県',
  '茨城県', '栃木県', '群馬県', '埼玉県', '千葉県', '東京都', '神奈川県',
  '新潟県', '富山県', '石川県', '福井県', '山梨県', '長野県', '岐阜県', '静岡県', '愛知県',
  '三重県', '滋賀県', '京都府', '大阪府', '兵庫県', '奈良県', '和歌山県',
  '鳥取県', '島根県', '岡山県', '広島県', '山口県',
  '徳島県', '香川県', '愛媛県', '高知県',
  '福岡県', '佐賀県', '長崎県', '熊本県', '大分県', '宮崎県', '鹿児島県', '沖縄県',
] as const

/**
 * 地方ブロックと所属都道府県のマッピング（イベント検索用）
 * キーが地方ブロック名、値が所属都道府県の配列
 */
export const REGION_MAP = {
  '北海道': ['北海道'],
  '東北': ['青森県', '岩手県', '宮城県', '秋田県', '山形県', '福島県'],
  '関東': ['茨城県', '栃木県', '群馬県', '埼玉県', '千葉県', '東京都', '神奈川県'],
  '中部': ['新潟県', '富山県', '石川県', '福井県', '山梨県', '長野県', '岐阜県', '静岡県', '愛知県'],
  '近畿': ['三重県', '滋賀県', '京都府', '大阪府', '兵庫県', '奈良県', '和歌山県'],
  '中国': ['鳥取県', '島根県', '岡山県', '広島県', '山口県'],
  '四国': ['徳島県', '香川県', '愛媛県', '高知県'],
  '九州・沖縄': ['福岡県', '佐賀県', '長崎県', '熊本県', '大分県', '宮崎県', '鹿児島県', '沖縄県']
} as const

/** 地方ブロック名の型 */
export type RegionName = keyof typeof REGION_MAP
// 型の中身: '北海道' | '東北' | '関東' | '中部' | '近畿' | '中国' | '四国' | '九州・沖縄'

/** 地方ブロック名の配列 */
export const REGION_NAME_LIST = Object.keys(REGION_MAP) as RegionName[]

/**
 * 地方ブロックから都道府県リストを取得
 * 例: getPrefecturesByRegion("関東") → ["茨城県", "栃木県", ...]
 */
export function getPrefecturesByRegion(region: RegionName): string[] {
  const prefectures = REGION_MAP[region]
  return prefectures ? [...prefectures] : []
}

/**
 * 都道府県から地方ブロック名を取得
 * 例: getRegionNameByPrefecture("東京都") → "関東"
 */
```

```

*/
export function getRegionNameByPrefecture(prefecture: string): RegionName | null {
  for (const [region, prefectures] of Object.entries(REGION_MAP)) {
    if ((prefectures as readonly string[]).includes(prefecture)) {
      return region as RegionName
    }
  }
  return null
}

```

期待される動作:

```

getPrefecturesByRegion('関東')
// → ['茨城県', '栃木県', '群馬県', '埼玉県', '千葉県', '東京都', '神奈川県']

getRegionNameByPrefecture('東京都')
// → '関東'

getRegionNameByPrefecture('不正な値')
// → null

```

as const の重要性:

```

// as const なし → 型は string[]
const regions = ['北海道', '東北', '関東']
// regions[0] の型: string

// as const あり → 型は readonly ["北海道", "東北", "関東"]
const regionsConst = ['北海道', '東北', '関東'] as const
// regionsConst[0] の型: "北海道"

```

実装しないとどうなる？

- 地方名と都道府県の対応を各コンポーネントにハードコードすると、修正箇所が散在してバグの温床になります
- **as const** なしだと、TypeScriptがタイプミスを検出できず、実行時にフィルタが動かないバグが起きます
- ユーティリティ関数がないと、地方から都道府県を取得するロジックがフィルターコンポーネントと Server Actionsの両方に重複します

17.4 イベントCRUD (Server Actions)

このセクションで学ぶこと

- CRUD (Create/Read/Update/Delete) 操作の実装パターン
- Server Actionsによるフォーム処理の流れ
- Zodによるバリデーション (入力値検証) の実装
- 権限チェック (認証・認可) の重要性
- `revalidatePath` によるキャッシュ更新の仕組み

Server Actions の実装

ファイル: `lib/actions/event.ts`

```
// lib/actions/event.ts
'use server'

import { z } from 'zod'
import { prisma } from '@lib/db'
import { auth } from '@lib/auth'
import { revalidatePath } from 'next/cache'
import { getPrefecturesByRegion, type RegionName as Region } from '@lib/prefectures'
import { UPCOMING_EVENTS_LIMIT } from '@lib/constants/limits'
import {
  ERR_AUTH_REQUIRED,
  ERR_EVENT_NOT_FOUND,
  ERR_INVALID_INPUT,
  ERR_PERMISSION_DENIED,
  ERR_EDIT_PERMISSION_DENIED,
} from '@lib/constants/errors'
import { sanitizeText, sanitizeUrl } from '@lib/sanitize'
import { requireActiveUser } from '@lib/actions/utlis'
import { getStartOfDay } from '@lib/utlis'

// =====
// バリデーションスキーマ（Zodで定義）
// =====

const eventSchema = z.object({
  title: z.string().min(1, 'タイトルを入力してください').max(100),
  startDate: z.string().min(1, '開始日を選択してください'),
  endDate: z.string().nullable().optional(),
  prefecture: z.string().min(1, '都道府県を選択してください'),
  city: z.string().nullable().optional(),
  venue: z.string().nullable().optional(),
  organizer: z.string().nullable().optional(),
  fee: z.string().nullable().optional(),
  description: z.string().nullable().optional(),
  externalUrl: z.string().nullable().optional(),
})

/**
 * イベントフォームデータを解析・バリデーションする。
 * createEvent と updateEvent で共通利用。
 */
function parseEventFormData(formData: FormData) {
  const hasSales = formData.get('hasSales') === 'true'

  const parsed = eventSchema.safeParse({
    title: formData.get('title') || '',
    startDate: formData.get('startDate') || '',
    endDate: formData.get('endDate') || null,
    prefecture: formData.get('prefecture') || '',
    city: formData.get('city') || null,
    venue: formData.get('venue') || null,
    organizer: formData.get('organizer') || null,
  })
}
```

```

    fee: formData.get('fee') || null,
    description: formData.get('description') || null,
    externalUrl: formData.get('externalUrl') || null,
  })

  if (!parsed.success) {
    return { error: parsed.error.issues[0]?.message || ERR_INVALID_INPUT }
  }

  const { title, startDate: startDateStr, endDate: endDateStr, prefecture,
    city, venue, organizer, fee, description, externalUrl } = parsed.data

  // 日付のパーズとバリデーション
  const startDate = new Date(startDateStr)
  if (isNaN(startDate.getTime())) {
    return { error: '開始日の形式が不正です' }
  }

  const endDate = endDateStr ? new Date(endDateStr) : null
  if (endDate && endDate < startDate) {
    return { error: '終了日は開始日以降を選択してください' }
  }

  return {
    data: {
      title: sanitizeText(title.trim()),
      startDate,
      endDate,
      prefecture,
      city: city ? sanitizeText(city.trim()) : null,
      venue: venue ? sanitizeText(venue.trim()) : null,
      organizer: organizer ? sanitizeText(organizer.trim()) : null,
      admissionFee: fee ? sanitizeText(fee.trim()) : null,
      hasSales,
      description: description ? sanitizeText(description.trim()) : null,
      externalUrl: externalUrl ? sanitizeUrl(externalUrl.trim()) : null,
    },
  }
}

```

処理の流れ（シーケンス）：

ブラウザ(フォーム) → Server Action

1. `requireActiveUser()` で認証チェック（ログイン済み？BANされていない？）
2. `parseEventFormData()` でバリデーション（Zodスキーマ + 日付前後チェック）
3. `sanitizeText()/sanitizeUrl()` でXSS対策のサニタイズ
4. `prisma.event.create()` でDB保存
5. `revalidatePath('/events')` でキャッシュ更新
6. `{ success: true, eventId }` を返す

Create (作成)

```
// lib/actions/event.ts (続き)

/** 新しいイベントを作成する。 */
export async function createEvent(formData: FormData) {
  // 認証チェック: ログイン済みかつアクティブなユーザーか確認
  const { userId, error } = await requireActiveUser('create_event')
  if (!userId) return { error: error! }

  // バリデーション: フォームデータの解析と検証
  const parsed = parseEventFormData(formData)
  if ('error' in parsed) {
    return { error: parsed.error }
  }

  // DB保存: イベントをデータベースに登録
  const event = await prisma.event.create({
    data: {
      createdBy: userId,
      ... parsed.data,
    },
  })

  // キャッシュ更新: イベント一覧ページを再検証
  revalidatePath('/events')

  return { success: true, eventId: event.id }
}
```

期待される動作:

```
// 正常系: イベントが作成される
const result = await createEvent(formData)
// → { success: true, eventId: "clxyz123 ..." }

// 異常系1: 未ログイン
// → { error: "認証が必要です" }

// 異常系2: タイトル未入力
// → { error: "タイトルを入力してください" }

// 異常系3: 終了日が開始日より前
// → { error: "終了日は開始日以降を選択してください" }
```


Read（読み取り） -- フィルタ付き一覧取得

```
// lib/actions/event.ts (続き)

/** イベント一覧をフィルター条件付きで取得する。 */
export async function getEvents(options?: {
  region?: string
  prefecture?: string
  showPast?: boolean
  month?: number
  year?: number
}) {
  const { region, prefecture, showPast = false, month, year } = options || {}
  const today = getStartOfToday()

  // 地方ブロック → 都道府県リストに変換
  let prefectureFilter: string[] | undefined
  if (prefecture) {
    prefectureFilter = [prefecture]
  } else if (region) {
    prefectureFilter = getPrefecturesByRegion(region as Region)
  }

  // 日付フィルタの構築
  let dateFilter: { gte?: Date; lt?: Date } | undefined
  if (month !== undefined && year !== undefined) {
    const startOfMonth = new Date(year, month, 1)
    const endOfMonth = new Date(year, month + 1, 1)
    dateFilter = { gte: startOfMonth, lt: endOfMonth }
  } else if (!showPast) {
    // 過去イベント非表示: 今日以降のイベントのみ
    dateFilter = { gte: today }
  }

  const events = await prisma.event.findMany({
    where: {
      isHidden: false,
      ... (dateFilter && { startDate: dateFilter }),
      ... (prefectureFilter && { prefecture: { in: prefectureFilter } }),
    },
    include: {
      creator: {
        select: { id: true, nickname: true, avatarUrl: true },
      },
    },
    orderBy: { startDate: 'asc' },
  })
}
```

```
    return { events }  
  }  
}
```

期待される動作:

```
// フィルタなし（今日以降のイベント全件）  
await getEvents()  
// → { events: [{ id: "...", title: "全国盆栽展", startDate: "2026-05-01", ... }, ...] }  
  
// 関東地方のみ  
await getEvents({ region: '関東' })  
// → { events: [東京都、埼玉県、千葉県 ... のイベントのみ] }  
  
// 過去イベントも含む  
await getEvents({ showPast: true })  
// → { events: [過去のイベントも含めた全件] }
```

Update（更新）と Delete（削除）

```
// lib/actions/event.ts（続き）

/** 既存のイベントを更新する。作成者のみ実行可能。 */
export async function updateEvent(eventId: string, formData: FormData) {
  const { userId, error } = await requireActiveUser('update_event')
  if (!userId) return { error: error! }

  const existingEvent = await prisma.event.findUnique({
    where: { id: eventId },
    select: { createdBy: true },
  })

  if (!existingEvent) return { error: ERR_EVENT_NOT_FOUND }

  // 権限チェック：作成者以外は編集不可
  if (existingEvent.createdBy !== userId) {
    return { error: ERR_EDIT_PERMISSION_DENIED }
  }

  const parsed = parseEventFormData(formData)
  if ('error' in parsed) return { error: parsed.error }

  await prisma.event.update({
    where: { id: eventId },
    data: parsed.data,
  })

  revalidatePath('/events')
  revalidatePath(`/events/${eventId}`)
  return { success: true }
}

/** イベントを削除する。作成者のみ実行可能。 */
export async function deleteEvent(eventId: string) {
  const { userId, error } = await requireActiveUser('delete_event')
  if (!userId) return { error: error! }

  const event = await prisma.event.findUnique({
    where: { id: eventId },
    select: { createdBy: true },
  })

  if (!event) return { error: ERR_EVENT_NOT_FOUND }
  if (event.createdBy !== userId) return { error: ERR_PERMISSION_DENIED }

  await prisma.event.delete({ where: { id: eventId } })
  revalidatePath('/events')
```

```
return { success: true }  
}
```

実装しないとどうなる？

- バリデーションがないと、不正なデータ（空のタイトル、存在しない日付等）がDBに保存されます
- 権限チェックがないと、他人のイベントを誰でも編集・削除できてしまいます
- `revalidatePath` がないと、イベントを作成/更新しても一覧ページに反映されません（キャッシュが古いまま）
- `sanitizeText` / `sanitizeUrl` がないと、XSS攻撃（悪意あるスクリプトの注入）のリスクがあります

17.5 イベントフォーム

このセクションで学ぶこと

- 作成と編集を1つのフォームコンポーネントで兼用する方法
- `useTransition` による非同期送信中のローディング状態管理
- フォームデータの構築と Server Actions への送信

EventForm コンポーネント

ファイル: `components/event/EventForm.tsx`

```
// components/event/EventForm.tsx
'use client'

import { useState, useTransition } from 'react'
import { useRouter } from 'next/navigation'
import { createEvent, updateEvent } from '@lib/actions/event'
import { PREFECTURES } from '@lib/prefectures'
import { MAX_EVENT_TITLE_LENGTH } from '@lib/constants/limits'
import { FormError } from '@components/common/FormError'

interface EventFormProps {
  /** フォームのモード: 'create' (新規作成) または 'edit' (編集) */
  mode: 'create' | 'edit'
  /** 編集モード時の初期データ */
  initialData?: {
    id: string
    title: string
    startDate: Date
    endDate: Date | null
    prefecture: string | null
    city: string | null
    venue: string | null
    organizer: string | null
    fee: string | null
    hasSales: boolean
    description: string | null
    externalUrl: string | null
  }
}

export function EventForm({ mode, initialData }: EventFormProps) {
  const router = useRouter()
  const [isPending, startTransition] = useTransition()
  const [error, setError] = useState<string | null>(null)

  // 各フィールドの状態管理 (編集時はinitialDataから初期値を設定)
  const [title, setTitle] = useState(initialData?.title || '')
  const [startDate, setStartDate] = useState(
    formatDateForInput(initialData?.startDate || null)
  )
  const [endDate, setEndDate] = useState(
    formatDateForInput(initialData?.endDate || null)
  )
  const [prefecture, setPrefecture] = useState(initialData?.prefecture || '')
  // ... 他のフィールドも同様

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    setError(null)

    // FormDataオブジェクトを構築
```

```

const formData = new FormData()
formData.append('title', title)
formData.append('startDate', startDate)
if (endDate) formData.append('endDate', endDate)
formData.append('prefecture', prefecture)
// ... 他のフィールドも追加

startTransition(async () => {
  // モードに応じてServer Actionを呼び出し
  const result = mode === 'create'
    ? await createEvent(formData)
    : await updateEvent(initialData!.id, formData)

  if (result.error) {
    setError(result.error)
    return
  }

  // 成功時はイベント詳細ページにリダイレクト
  if (mode === 'create' && 'eventId' in result) {
    router.push(`/events/${result.eventId}`)
  } else {
    router.push(`/events/${initialData!.id}`)
  }
  router.refresh()
})
}

return (
  <form onSubmit={handleSubmit} className="space-y-6">
    <FormError message={error} />

    {/* タイトル入力（必須） */}
    <div>
      <label htmlFor="title" className="block text-sm font-medium mb-1">
        タイトル <span className="text-red-500">*</span>
      </label>
      <input
        id="title"
        type="text"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
        required
        maxLength={MAX_EVENT_TITLE_LENGTH}
        className="w-full px-3 py-2 border rounded-lg bg-background"
        placeholder="例：第30回 全国盆栽展"
      />
    </div>

    {/* 都道府県選択（必須） */}
    <select
      id="prefecture"
      value={prefecture}

```

```

      onChange={e => setPrefecture(e.target.value)}
      required
    >
      <option value="">選択してください</option>
      {PREFECTURES.map((pref) => (
        <option key={pref} value={pref}>{pref}</option>
      ))}
    </select>

    {/* 送信ボタン */}
    <button type="submit" disabled={isPending}>
      {isPending ? '保存中 ... ' : mode === 'create' ? '登録する' : '更新する'}
    </button>
  </form>
)
}

```

期待される表示:

新規登録時:

```

[タイトル *]      第30回 全国盆栽展
[開始日時 *]      2026-05-01T10:00
[終了日時]         2026-05-05T17:00
[都道府県 *]      埼玉県 ▼
[市区町村]        さいたま市
[会場名]           大宮盆栽美術館
[主催者]           全日本盆栽協会
[入場料]           500円
[☑ 即売あり]
[詳細説明]         盆栽の名品が一堂に ...
[外部リンク]       https://example.com

[キャンセル] [登録する]

```

新規登録ページ

ファイル: `app/(main)/events/new/page.tsx`

```
// app/(main)/events/new/page.tsx
import { redirect } from 'next/navigation'
import { auth } from '@/lib/auth'
import { EventForm } from '@/components/event/EventForm'

export const metadata = { title: 'イベントを登録 - BON-LOG' }

export default async function NewEventPage() {
  const session = await auth()

  // 未ログインの場合はログインページへリダイレクト
  if (!session?.user?.id) {
    redirect('/login')
  }

  return (
    <div className="max-w-2xl mx-auto space-y-6">
      <div className="bg-card rounded-lg border p-6">
        <h1 className="text-2xl font-bold mb-6">イベントを登録</h1>
        <EventForm mode="create" />
      </div>
    </div>
  )
}
```

編集ページ

ファイル: `app/(main)/events/[id]/edit/page.tsx`


```
// app/(main)/events/[id]/edit/page.tsx
import { notFound, redirect } from 'next/navigation'
import { auth } from '@lib/auth'
import { getEvent } from '@lib/actions/event'
import { EventForm } from '@components/event/EventForm'

export default async function EditEventPage({
  params,
}: {
  params: Promise<{ id: string }>
}) {
  const { id } = await params
  const session = await auth()

  if (!session?.user?.id) redirect('/login')

  const result = await getEvent(id)
  if (result.error || !result.event) notFound()

  const event = result.event

  // 所有者でない場合は詳細ページにリダイレクト
  if (!event.isOwner) redirect(`/events/${id}`)

  return (
    <div className="max-w-2xl mx-auto space-y-6">
      <div className="bg-card rounded-lg border p-6">
        <h1 className="text-2xl font-bold mb-6">イベントを編集</h1>
        <EventForm
          mode="edit"
          initialData={{
            id: event.id,
            title: event.title,
            startDate: event.startDate,
            endDate: event.endDate,
            prefecture: event.prefecture,
            city: event.city,
            venue: event.venue,
            organizer: event.organizer,
            fee: event.admissionFee,
            hasSales: event.hasSales,
            description: event.description,
            externalUrl: event.externalUrl,
          }}
        />
      </div>
    </div>
  )
}
```

実装しないとうなる？

- `mode` プロパティで作成/編集を切り替えないと、ほぼ同じフォームのコードが2箇所に重複します
- `useTransition` を使わないと、送信中にボタンを連打でき、イベントが二重登録されるリスクがあります
- 編集ページで所有者チェックがないと、URLを直接入力して他人のイベントを編集できてしまいます

17.6 イベント一覧ページとフィルタ

このセクションで学ぶこと

- Server ComponentでURLパラメータに基づくデータ取得
- 地域フィルターコンポーネントの実装（地方ブロック + 都道府県）
- 過去イベント表示切替の仕組み
- カレンダー/リスト表示の切替

イベント一覧ページ

ファイル: `app/(main)/events/page.tsx`

```
// app/(main)/events/page.tsx
import Link from 'next/link'
import { Suspense } from 'react'
import { getEvents } from '@lib/actions/event'
import { EventCalendarWrapper } from '@components/event/EventCalendarWrapper'
import { EventList } from '@components/event/EventList'
import { RegionFilter } from '@components/event/RegionFilter'
import { ShowPastToggle } from '@components/event/ShowPastToggle'
import { EventFilterPersistence } from '@components/event/EventFilterPersistence'
import { Plus as PlusIcon, Calendar as CalendarIcon, List as ListIcon } from 'lucide-react'

export const metadata = { title: 'イベント - BON-LOG' }

interface EventsPageProps {
  searchParams: Promise<{
    region?: string // 地方ブロック
    prefecture?: string // 都道府県
    view?: string // 表示モード (calendar / list)
    showPast?: string // 過去イベント表示フラグ
    year?: string // カレンダー表示年
    month?: string // カレンダー表示月 (1-12)
  }>
}

export default async function EventsPage({ searchParams }: EventsPageProps) {
  const params = await searchParams
  const view = params.view || 'calendar'
  const showPast = params.showPast === 'true'

  // フィルター条件を適用してイベントを取得
  const { events } = await getEvents({
    region: params.region,
    prefecture: params.prefecture,
    showPast,
  })

  return (
    <div className="space-y-6">
      {/* フィルター設定の永続化 (UIなし) */}
      <Suspense fallback={null}>
        <EventFilterPersistence />
      </Suspense>

      {/* ハッダー */}
      <div className="flex items-center justify-between">
        <h1 className="text-2xl font-bold">イベント</h1>
        <Link href="/events/new" className="flex items-center gap-2 px-4 py-2 bg-primary text-white">
          <PlusIcon className="w-4 h-4" />
          <span>イベントを登録</span>
        </Link>
      </div>
    </div>
  )
}
```

```

{ /* 地域フィルター */}
<Suspense fallback={<div className="h-40 bg-muted animate-pulse rounded-lg" />}>
  <RegionFilter currentRegion={params.region} currentPrefecture={params.prefecture}>
</Suspense>

{ /* 表示切替: カレンダー/リスト + 過去イベントトグル */}
<div className="flex items-center justify-between">
  <div className="flex items-center gap-2">
    <Link href={` /events?${new URLSearchParams({ ... params, view: 'calendar' }).toString()}>
      <div className="flex items-center gap-1 px-3 py-1.5 rounded-lg ${view === 'calendar' ? 'bg-white' : 'bg-white/50'}>
        <CalendarIcon className="w-4 h-4" /> <span className="text-sm">カレンダー</span>
      </div>
    </Link>
    <Link href={` /events?${new URLSearchParams({ ... params, view: 'list' }).toString()}>
      <div className="flex items-center gap-1 px-3 py-1.5 rounded-lg ${view === 'list' ? 'bg-white' : 'bg-white/50'}>
        <ListIcon className="w-4 h-4" /> <span className="text-sm">リスト</span>
      </div>
    </Link>
  </div>
  <ShowPastToggle showPast={showPast} />
</div>

{ /* イベント表示 */}
{view === 'calendar' ? (
  <EventCalendarWrapper
    events={events}
    initialYear={params.year ? parseInt(params.year, 10) : undefined}
    initialMonth={params.month ? parseInt(params.month, 10) : undefined}
  />
) : (
  <div>
    <h2 className="text-lg font-semibold mb-4">
      イベント一覧
    </h2>
    <span className="text-sm font-normal text-muted-foreground ml-2">({events.length} イベント)</span>
    <EventList events={events} emptyMessage="該当するイベントがありません" />
  </div>
)}
</div>
)
}

```

期待される表示:

[イベント]

[+ イベントを登録]

地域で絞り込み

地方

[北海道] [東北] [関東(選択中)] [中部] [近畿] ...

都道府県

[すべての都道府県 ▼]

フィルターをクリア

[カレンダー(選択中)] [リスト]

[☑ 終了イベントも表示]

2026年5月

◀ 2026年5月 [今日] ▶

日 月 火 水 木 金 土

... 1 2 3

[第30回全国盆栽展]

4 5 6 7 8 9 10

...

地域フィルターコンポーネント

ファイル: `components/event/RegionFilter.tsx`

```
// components/event/RegionFilter.tsx
'use client'

import { useRouter, useSearchParams } from 'next/navigation'
import { REGION_NAME_LIST as REGION_LIST, PREFECTURES } from '@/lib/prefectures'

interface RegionFilterProps {
  currentRegion?: string
  currentPrefecture?: string
}

export function RegionFilter({ currentRegion, currentPrefecture }: RegionFilterProps) {
  const router = useRouter()
  const searchParams = useSearchParams()

  // 地方ブロック選択時: 都道府県フィルターは解除 (排他的)
  const handleRegionChange = (region: string) => {
    const params = new URLSearchParams(searchParams.toString())
    if (region) {
      params.set('region', region)
      params.delete('prefecture')
    } else {
      params.delete('region')
    }
    router.push(`/events?${params.toString()}`)
  }

  // 都道府県選択時: 地方フィルターは解除 (排他的)
  const handlePrefectureChange = (prefecture: string) => {
    const params = new URLSearchParams(searchParams.toString())
    if (prefecture) {
      params.set('prefecture', prefecture)
      params.delete('region')
    } else {
      params.delete('prefecture')
    }
    router.push(`/events?${params.toString()}`)
  }

  // フィルタークリア: 地方と都道府県の両方を解除
  const handleClear = () => {
    const params = new URLSearchParams(searchParams.toString())
    params.delete('region')
    params.delete('prefecture')
    router.push(`/events?${params.toString()}`)
  }

  return (
    <div className="bg-card rounded-lg border p-4 space-y-4">
      <h3 className="font-semibold">地域で絞り込み</h3>

```

```

    { /* 地方ブロックボタン */}
    <div>
      <label className="block text-sm text-muted-foreground mb-2">地方</label>
      <div className="flex flex-wrap gap-2">
        {REGION_LIST.map((region) => (
          <button
            key={region}
            onClick={() => handleRegionChange(currentRegion === region ? '' : region)}
            className={`px-3 py-1 text-sm rounded-full border transition-colors ${
              currentRegion === region
                ? 'bg-primary text-primary-foreground border-primary'
                : 'hover:bg-muted'
            }`}
          >
            {region}
          </button>
        ))}
      </div>
    </div>

    { /* 都道府県セレクトボックス */}
    <div>
      <label htmlFor="prefecture-select" className="block text-sm text-muted-foreground">
        都道府県
      </label>
      <select
        id="prefecture-select"
        value={currentPrefecture || ''}
        onChange={(e) => handlePrefectureChange(e.target.value)}
        className="w-full px-3 py-2 border rounded-lg bg-background"
      >
        <option value=""><すべての都道府県</option>
        {PREFECTURES.map((pref) => (
          <option key={pref} value={pref}>{pref}</option>
        ))}
      </select>
    </div>

    { /* クリアボタン（フィルター適用中のみ表示） */}
    {(currentRegion || currentPrefecture) && (
      <button onClick={handleClear} className="text-sm text-muted-foreground hover:text-foreground">
        フィルターをクリア
      </button>
    )}
  </div>
)
}

```

ポイント: 地方ブロックと都道府県は排他的

地方ブロック「関東」を選択すると、自動的に都道府県フィルターがクリアされます。逆に都道府県「東京都」を選択すると、地方ブロックの選択がクリアされます。これにより、フィルタの重複適用による混乱を防ぎます。

実装しないとうなる？

- 地方と都道府県を同時に適用すると、「関東の東京都」という意味のない二重フィルタになります
- URLパラメータに反映しないと、ブラウザの「戻る」ボタンでフィルタ状態が戻りません
- `useSearchParams` で既存パラメータを維持しないと、フィルタ操作のたびに他のパラメータ（view等）が消えます

17.7 過去イベント表示切替

このセクションで学ぶこと

- チェックボックス形式のURLパラメータ切替の実装
- Linkコンポーネントを使った状態切替（formなしの方法）

ShowPastToggle コンポーネント

ファイル: `components/event/ShowPastToggle.tsx`


```
// components/event/ShowPastToggle.tsx
'use client'

import Link from 'next/link'
import { useSearchParams } from 'next/navigation'

interface ShowPastToggleProps {
  showPast: boolean
}

export function ShowPastToggle({ showPast }: ShowPastToggleProps) {
  const searchParams = useSearchParams()

  // 切替後のURLパラメータを構築
  const newParams = new URLSearchParams(searchParams.toString())
  if (showPast) {
    // 現在表示中 → パラメータを削除して非表示に切替
    newParams.delete('showPast')
  } else {
    // 現在非表示 → パラメータを追加して表示に切替
    newParams.set('showPast', 'true')
  }

  return (
    <Link>
      href={`\`/events?${newParams.toString()}`}
      className="flex items-center gap-2 text-sm cursor-pointer"
    >
      <input
        type="checkbox"
        checked={showPast}
        readOnly
        className="w-4 h-4 rounded cursor-pointer"
      />
      <span className="text-muted-foreground hover:text-foreground whitespace-nowrap">
        終了イベントも表示
      </span>
    </Link>
  )
}
```

期待される動作:

初期状態 (`showPast=false`) :

URL: `/events?region=関東`

表示: `[☐ 終了イベントも表示]` ← チェックなし

結果: 今日以降のイベントのみ表示

クリック後 (`showPast=true`) :

URL: `/events?region=関東&showPast=true`

表示: `[☒ 終了イベントも表示]` ← チェックあり

結果: 過去のイベントも含めて表示 (終了済みは半透明)

設計ポイント:

- `<Link>` で囲むことで、チェックボックスの状態変更がURLパラメータの変更と連動
- `readOnly` 属性でチェックボックス自体の直接操作を防止 (Linkクリックで制御)
- 既存の `searchParams` を維持しつつ、`showPast` のみを切り替え

実装しないとどうなる？

- 過去イベント表示の状態がURLに反映されないと、ページリロードで元に戻ってしまいます
- `readOnly` なしだと、チェックボックスの状態とURLが不整合になることがあります

17.8 フィルター永続化

このセクションで学ぶこと

- `localStorage`を使ったフィルター設定の永続化
- UIを持たないユーティリティコンポーネントの設計パターン
- `useEffect` による保存と復元の仕組み

EventFilterPersistence コンポーネント

ファイル: `components/event/EventFilterPersistence.tsx`

```
// components/event/EventFilterPersistence.tsx
'use client'

import { useEffect } from 'react'
import { useRouter, useSearchParams } from 'next/navigation'

const STORAGE_KEY = 'event-filter-settings'

interface FilterSettings {
  region?: string
  prefecture?: string
  view?: string
  showPast?: string
}

export function EventFilterPersistence() {
  const router = useRouter()
  const searchParams = useSearchParams()

  // Effect 1: URLパラメータの変更を監視してlocalStorageに保存
  useEffect(() => {
    const currentSettings: FilterSettings = {}
    const region = searchParams.get('region')
    const prefecture = searchParams.get('prefecture')
    const view = searchParams.get('view')
    const showPast = searchParams.get('showPast')

    if (region) currentSettings.region = region
    if (prefecture) currentSettings.prefecture = prefecture
    if (view) currentSettings.view = view
    if (showPast) currentSettings.showPast = showPast

    // パラメータが1つ以上ある場合のみ保存
    if (Object.keys(currentSettings).length > 0) {
      localStorage.setItem(STORAGE_KEY, JSON.stringify(currentSettings))
    }
  }, [searchParams])

  // Effect 2: 初回アクセス時にlocalStorageから設定を復元
  useEffect(() => {
    const hasParams = searchParams.toString().length > 0

    if (!hasParams) {
      try {
        const saved = localStorage.getItem(STORAGE_KEY)
        if (saved) {
          const settings: FilterSettings = JSON.parse(saved)
          const params = new URLSearchParams()

          if (settings.region) params.set('region', settings.region)
          if (settings.prefecture) params.set('prefecture', settings.prefecture)
        }
      } catch {}
    }
  })
}
```

```

    if (settings.view) params.set('view', settings.view)
    if (settings.showPast) params.set('showPast', settings.showPast)

    if (params.toString()) {
      router.replace(`/events?${params.toString()}`)
    }
  } catch {
    // localStorageのエラー（プライベートモード等）は無視
  }
}
}, []) // eslint-disable-line react-hooks/exhaustive-deps

// UIを持たないコンポーネント
return null
}

```

期待される動作:

1回目のアクセス:

ユーザーが `/events` にアクセス → URLパラメータなし → `localStorage`空 → デフォルト表示

ユーザーが関東を選択:

URL: `/events?region=関東`
`localStorage`: `{"region": "関東"}`

ブラウザを閉じて再度 `/events` にアクセス:

URLパラメータなし → `localStorage`から復元 → `/events?region=関東` に自動遷移
 → 前回の設定が復元される！

設計ポイント:

- `return null` でUIを一切レンダリングしない「ユーティリティコンポーネント」パターン
- `Suspense` で囲む必要がある（`useSearchParams` が`Suspense`バウンダリを要求するため）
- 第2のEffectで依存配列を空にしているのは、初回マウント時のみ実行したいため（無限ループ防止）
- `router.replace` で履歴に残さず置換（「戻る」ボタンで余計なエントリが増えない）

実装しないとどうなる？

- 毎回サイトを開くたびに、最初から地域フィルターを設定し直す必要があります
- ユーザーが「いつも関東のイベントを見る」のような使い方をするとき、非常に不便です

17.9 カレンダー表示

このセクションで学ぶこと

- date-fnsを使った月間カレンダーの実装
- 動的インポート (`next/dynamic`) によるSSR無効化
- 複数日にまたがるイベントの表示処理
- タイムゾーンの考慮 (UTCずれ防止)

EventCalendar コンポーネント

ファイル: `components/event/EventCalendar.tsx`

```
// components/event/EventCalendar.tsx
'use client'

import { useState, useCallback } from 'react'
import Link from 'next/link'
import { useRouter, useSearchParams } from 'next/navigation'
import {
  format, startOfMonth, endOfMonth, startOfWeek, endOfWeek,
  eachDayOfInterval, isSameMonth, addMonths, subMonths, isToday,
} from 'date-fns'
import { ja } from 'date-fns/locale'
import { CALENDAR_EVENTS_PER_DAY } from '@lib/constants/limits'

interface EventCalendarProps {
  events: Event[]
  onMonthChange?: (year: number, month: number) => void
  initialYear?: number
  initialMonth?: number // 1-12
}

export function EventCalendar({ events, onMonthChange, initialYear, initialMonth }: EventCalendarProps) {
  const router = useRouter()
  const searchParams = useSearchParams()

  // 初期表示月の計算
  const getInitialDate = useCallback(() => {
    if (initialYear && initialMonth) {
      return new Date(initialYear, initialMonth - 1, 1)
    }
    return new Date()
  }, [initialYear, initialMonth])

  const [currentMonth, setCurrentMonth] = useState(getInitialDate())

  // URLパラメータ更新（年月をURLに反映）
  const updateUrlParams = useCallback((year: number, month: number) => {
    const params = new URLSearchParams(searchParams.toString())
    params.set('year', year.toString())
    params.set('month', (month + 1).toString())
    router.replace(`/events?${params.toString()}`, { scroll: false })
  }, [router, searchParams])

  // カレンダー表示用の日付計算
  const monthStart = startOfMonth(currentMonth)
  const monthEnd = endOfMonth(currentMonth)
  const calendarStart = startOfWeek(monthStart)
  const calendarEnd = endOfWeek(monthEnd)
  const days = eachDayOfInterval({ start: calendarStart, end: calendarEnd })

  /**
   * タイムゾーンずれ防止: ISO文字列からYYYY-MM-DDを直接抽出
   */
}
```

```

* 例: "2026-05-03T15:00:00.000Z" → "2026-05-03"
*/
const getDateString = (date: Date | string): string ⇒ {
  const isoString = typeof date === 'string' ? date : date.toISOString()
  const match = isoString.match(/^(\d{4}-\d{2}-\d{2})/)
  if (match) return match[1]
  return format(new Date(date), 'yyyy-MM-dd')
}

// 指定した日に開催されているイベントを取得（複数日にまたがるイベントも考慮）
const getEventsForDay = (day: Date) ⇒ {
  const dayStr = format(day, 'yyyy-MM-dd')
  return events.filter((event) ⇒ {
    const startStr = getDateString(event.startDate)
    const endStr = event.endDate ? getDateString(event.endDate) : startStr
    return dayStr ≥ startStr && dayStr ≤ endStr
  })
}

return (
  <div className="bg-card rounded-lg border">
    {/* ヘッダー: ◀ 2026年5月 [今日] ▶ */}
    <div className="flex items-center justify-between p-4 border-b">
      <button onClick={handlePrevMonth} aria-label="前月">◀</button>
      <div className="flex items-center gap-4">
        <h2 className="text-lg font-semibold">
          {format(currentMonth, 'yyyy年M月', { locale: ja })}
        </h2>
        <button onClick={handleToday}>今日</button>
      </div>
      <button onClick={handleNextMonth} aria-label="次月">▶</button>
    </div>

    {/* 曜日ヘッダー */}
    <div className="grid grid-cols-7 border-b">
      {[ '日', '月', '火', '水', '木', '金', '土' ].map((day, i) ⇒ (
        <div key={day} className={`text-center text-sm py-2 ${
          i === 0 ? 'text-red-500' : i === 6 ? 'text-blue-500' : ''
        }`} >{day}</div>
      ))}
    </div>

    {/* 日付グリッド */}
    <div className="grid grid-cols-7">
      {days.map((day) ⇒ {
        const dayEvents = getEventsForDay(day)
        return (
          <div key={day.toISOString()} className="min-h-[100px] p-1 border-b border-r">
            <div className={`text-sm w-7 h-7 flex items-center justify-center rounded-fu
              isToday(day) ? 'bg-primary text-primary-foreground' : ''
            }`} >
              {format(day, 'd')}
            </div>
          </div>
        )
      })}
    </div>
  </div>
)

```

```

    { /* イベント表示 (最大3件) */
    {dayEvents.slice(0, CALENDAR_EVENTS_PER_DAY).map((event) => (
      <Link key={event.id} href={`/${events}/${event.id}`}
        className="block text-xs bg-primary/10 text-primary px-1 py-0.5 rounded
        {event.title}
      </Link>
    ))}
    {dayEvents.length > CALENDAR_EVENTS_PER_DAY && (
      <div className="text-xs text-muted-foreground px-1">
        +{dayEvents.length - CALENDAR_EVENTS_PER_DAY}件
      </div>
    )}
  </div>
)
}}
</div>
</div>
)
}

```

定数値:

- `CALENDAR_EVENTS_PER_DAY = 3` -- 1日のセルに表示するイベント数の上限
(`lib/constants/limits.ts` で定義)

SSR無効化ラッパー

ファイル: `components/event/EventCalendarWrapper.tsx`


```
// components/event/EventCalendarWrapper.tsx
'use client'

import dynamic from 'next/dynamic'

// SSR無効化: カレンダーはクライアントサイドのみでレンダリング
const EventCalendar = dynamic(
  () => import('./EventCalendar').then((mod) => mod.EventCalendar),
  {
    ssr: false, // サーバーサイドレンダリングを無効化
    loading: () => (
      <div className="bg-card rounded-lg border p-4">
        <div className="h-[500px] bg-muted animate-pulse rounded-lg" />
      </div>
    ),
  }
)

export function EventCalendarWrapper({ events, initialYear, initialMonth }: EventCalendarProps) {
  return <EventCalendar events={events} initialYear={initialYear} initialMonth={initialMonth} />
}
```

なぜSSRを無効化するのか？

EventCalendarコンポーネントは `new Date()` で現在日時を取得し、`useState` で月の状態を管理します。サーバーとクライアントで `new Date()` の結果が異なると、ハイドレーションの不一致エラーが発生します。`ssr: false` にすることで、カレンダーはクライアントでのみレンダリングされ、この問題を回避できます。

実装しないとどうなる？

- SSR無効化しないと、「サーバーとクライアントのHTMLが一致しません」というハイドレーションエラーが発生します
- `loading` を指定しないと、カレンダーの読み込み中にレイアウトがガタつきます (CLS問題)
- タイムゾーン処理がないと、日本時間で5月3日のイベントが5月2日に表示される場合があります

17.10 イベントカードとリスト表示

このセクションで学ぶこと

- イベントカードの状態表示（終了/開催中/即売あり）
- date-fnsによる日本語日付フォーマット
- レスポンシブグリッドレイアウト

EventCard コンポーネント

ファイル: `components/event/EventCard.tsx`

```
// components/event/EventCard.tsx
import Link from 'next/link'
import { format } from 'date-fns'
import { ja } from 'date-fns/locale'

/**
 * イベント日程をフォーマットする関数
 * 単日: "5月1日(水)"
 * 複数日: "5月1日(水) ~ 5月5日(日)"
 */
function formatEventDate(startDate: Date, endDate: Date | null): string {
  const start = new Date(startDate)
  const startStr = format(start, 'M月d日(E)', { locale: ja })
  if (!endDate) return startStr

  const end = new Date(endDate)
  if (start.toDateString() === end.toDateString()) return startStr

  const endStr = format(end, 'M月d日(E)', { locale: ja })
  return `${startStr} ~ ${endStr}`
}

export function EventCard({ event }: EventCardProps) {
  const now = new Date()
  const startDate = new Date(event.startDate)
  const endDate = event.endDate ? new Date(event.endDate) : null

  // ステータス判定
  const isEnded = endDate ? endDate < now : startDate < now
  const isOngoing = !isEnded && startDate ≤ now && endDate && endDate ≥ now

  return (
    <Link
      href={` /events/${event.id}`}
      className={`block bg-card border rounded-lg p-4 hover:bg-muted/50 ${isEnded ? 'opacity-50'}`}>
      <h3 className="font-semibold mb-2 line-clamp-2">{event.title}</h3>

      <div className="space-y-1.5 text-sm text-muted-foreground">
        { /* 日程 */ }
        <div className="flex items-center gap-2">
          <CalendarIcon className="w-4 h-4" />
          <span>{formatEventDate(event.startDate, event.endDate)}</span>
        </div>
        { /* 場所 */ }
        {(event.prefecture || event.city || event.venue) && (
          <div className="flex items-center gap-2">
            <MapPinIcon className="w-4 h-4" />
            <span>{event.prefecture}{event.city && ` ${event.city}`} {event.venue && ` / ${event.venue}`}</span>
          </div>
        )}
      </div>
    </Link>
  )
}
```

```

</div>

{/* ステータスバッジ */}
<div className="flex items-center gap-2 mt-3">
  {event.admissionFee && (
    <span className="text-xs px-2 py-0.5 bg-muted rounded-full">{event.admissionFee}</span>
  )}
  {event.hasSales && (
    <span className="text-xs px-2 py-0.5 bg-primary/10 text-primary rounded-full">即売会</span>
  )}
  {isEnded && (
    <span className="text-xs px-2 py-0.5 bg-muted-foreground/20 text-muted-foreground rounded-full">終了</span>
  )}
  {isOngoing && (
    <span className="text-xs px-2 py-0.5 bg-green-500/10 text-green-600 rounded-full">開催中</span>
  )}
</div>
</Link>
)
}

```

期待される表示:

第30回 全国盆栽展

📅 5月1日(木) ~ 5月5日(月)

📍 埼玉県 さいたま市 / 大宮盆栽美術館

¥500 [即売あり] [開催中]

春の盆栽講座

📅 4月15日(火)

📍 東京都 / 上野恩賜公園

[無料] [終了]

← opacity-60 (半透明)

EventList コンポーネント

ファイル: `components/event/EventList.tsx`

```
// components/event/EventList.tsx
import { EventCard } from './EventCard'

export function EventList({ events, emptyMessage = 'イベントがありません' }: EventListProps)
  if (events.length === 0) {
    return (
      <div className="text-center py-12 text-muted-foreground">
        {emptyMessage}
      </div>
    )
  }

  return (
    <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-1 xl:grid-cols-2">
      {events.map((event) => (
        <EventCard key={event.id} event={event} />
      ))}
    </div>
  )
}
```

レスポンス設計:

- モバイル（デフォルト）：1列
- タブレット（640px以上）：2列
- ラップトップ（1024px以上）：1列（サイドバーとの組み合わせ考慮）
- ワイドスクリーン（1280px以上）：2列

実装しないとどうなる？

- ステータス表示がないと、ユーザーは終了済みイベントに申し込もうとする可能性があります
- 終了イベントの半透明処理がないと、有効なイベントと終了イベントが見分けにくくなります
- 空の状態の処理がないと、イベント0件でも何も表示されず、ユーザーは「読み込み中？」と混乱します

17.11 イベント詳細ページ

このセクションで学ぶこと

- 動的メタデータ生成 (`generateMetadata`)
- SEO構造化データ (JSON-LD) の出力
- パンくずリストの実装
- 所有者判定による編集・削除ボタンの出し分け

イベント詳細ページ

ファイル: `app/(main)/events/[id]/page.tsx`

```
// app/(main)/events/[id]/page.tsx
import { notFound } from 'next/navigation'
import { Metadata } from 'next'
import Link from 'next/link'
import Image from 'next/image'
import { format } from 'date-fns'
import { ja } from 'date-fns/locale'
import { getEvent } from '@lib/actions/event'
import { DeleteEventButton } from './DeleteEventButton'
import { EventJsonLd } from '@components/seo/JsonLd'
import { Breadcrumb } from '@components/common/Breadcrumb'

// 動的メタデータ: イベント情報に基づいたSEO用メタデータ
export async function generateMetadata({ params }: EventDetailPageProps): Promise<Metadata> {
  const { id } = await params
  const result = await getEvent(id)

  if (result.error || !result.event) {
    return { title: 'イベントが見つかりません' }
  }

  const event = result.event
  const startDateStr = format(new Date(event.startDate), 'yyyy年M月d日', { locale: ja })
  const locationStr = [event.prefecture, event.city, event.venue].filter(Boolean).join(' ')
  const description = `${startDateStr}開催${locationStr ? ` (${locationStr})` : ''}`

  return {
    title: event.title,
    description,
    openGraph: {
      type: 'website',
      title: `${event.title} - 盆栽イベント`,
      description,
    },
  }
}

export default async function EventDetailPage({ params }: EventDetailPageProps) {
  const { id } = await params
  const result = await getEvent(id)

  if (result.error || !result.event) notFound()

  const event = result.event
  const now = new Date()
  const isEnded = event.endDate ? new Date(event.endDate) < now : new Date(event.startDate) > now
  const isOngoing = !isEnded && new Date(event.startDate) ≤ now &&
    event.endDate && new Date(event.endDate) ≥ now

  return (
    <

```

```

{ /* パンくずリスト */}
<Breadcrumb items={[
  { name: 'ホーム', href: '/feed' },
  { name: 'イベント', href: '/events' },
  { name: event.title },
]} />

{ /* SEO用JSON-LD */}
<EventJsonLd
  name={event.title}
  startDate={new Date(event.startDate).toISOString()}
  endDate={event.endDate ? new Date(event.endDate).toISOString() : undefined}
/>

<div className="space-y-6">
  <Link href="/events">< イベント一覧に戻る</Link>

  <div className="bg-card rounded-lg border p-6">
    { /* ステータスバッジ + タイトル */}
    <div className="flex items-start justify-between gap-4 mb-6">
      <div>
        <div className="flex items-center gap-2 mb-2">
          {isEnded && <span className="text-xs px-2 py-0.5 bg-muted-foreground/20 rounded">終了</span>}
          {isOngoing && <span className="text-xs px-2 py-0.5 bg-green-500/10 text-green-800">開催中</span>}
          {event.hasSales && <span className="text-xs px-2 py-0.5 bg-primary/10 text-white">販売中</span>}
        </div>
        <h1 className="text-2xl font-bold">{event.title}</h1>
      </div>

      { /* 所有者用アクションボタン */}
      {event.isOwner && (
        <div className="flex items-center gap-2">
          <Link href={`/${event.id}/edit`} className="border rounded-lg px-3 py-2">編集</Link>
          <DeleteEventButton eventId={event.id} />
        </div>
      )}
    </div>

    { /* 詳細情報 */}
    <div className="space-y-4">
      <p>📅 {format(event.startDate, 'yyyy年M月d日(E) HH:mm', { locale: ja })}</p>
      <p>📍 {event.prefecture}{event.city && ` ${event.city}`}</p>
      <p>👤 主催: {event.organizer}</p>
      <p>💰 入場料: {event.admissionFee}</p>
      {event.externalUrl && <a href={event.externalUrl} target="_blank" rel="noopener">詳細を見る</a>}
    </div>

    { /* 詳細説明 */}
    {event.description && (
      <div className="mt-6 pt-6 border-t">
        <h2 className="font-semibold mb-3">詳細</h2>
        <p className="whitespace-pre-wrap">{event.description}</p>
      </div>
    )}
  </div>

```



```

    })

    { /* 登録者情報 */ }
    <div className="mt-6 pt-6 border-t">
      <p className="text-sm text-muted-foreground">
        登録者:
        <Link href={`\ /users/${event.creator.id}`} >{event.creator.nickname}</Link>
      </p>
    </div>
  </div>
</div>
</>
)
}

```

`event.isOwner` の仕組み:

`getEvent` 関数内で、現在のセッションユーザーIDとイベントの `createdBy` を比較し、結果を `isOwner` プロパティとして返しています。

```

// lib/actions/event.ts (getEvent関数内)
return {
  event: {
    ...event,
    isOwner: currentUserId === event.createdBy,
  },
}

```

実装しないとどうなる？

- `generateMetadata` がないと、SNSでイベントURLをシェアしたとき「無題のページ」と表示されます
- JSON-LDがないと、Google検索でイベント情報がリッチスニペットとして表示されません
- `isOwner` チェックがないと、全ユーザーに編集・削除ボタンが表示されてしまいます

17.12 イベント削除

このセクションで学ぶこと

- 2段階確認プロセス（誤操作防止）の実装
- 削除ボタンのUI/UXパターン

DeleteEventButton コンポーネント

ファイル: `components/event/DeleteEventButton.tsx`

```
// components/event/DeleteEventButton.tsx
'use client'

import { useState } from 'react'
import { useRouter } from 'next/navigation'
import { Button } from '@components/ui/button'
import { deleteEvent } from '@lib/actions/event'
import { Trash2 } from 'lucide-react'

export function DeleteEventButton({ eventId }: { eventId: string }) {
  const [showConfirm, setShowConfirm] = useState(false)
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState<string | null>(null)
  const router = useRouter()

  async function handleDelete() {
    setLoading(true)
    setError(null)
    try {
      const result = await deleteEvent(eventId)
      if (result && 'error' in result) {
        setError(result.error ?? '削除に失敗しました')
        setLoading(false)
        return
      }
      router.push('/events')
      router.refresh()
    } catch {
      setError('イベントの削除に失敗しました')
      setLoading(false)
    }
  }

  // 確認ダイアログ表示中
  if (showConfirm) {
    return (
      <div className="space-y-3" role="dialog" aria-label="削除確認">
        <p className="text-sm text-destructive font-medium">
          このイベントを削除しますか？この操作は取り消せません。
        </p>
        {error && <p className="text-sm text-destructive">{error}</p>}
        <div className="flex gap-2">
          <Button variant="destructive" size="sm" onClick={handleDelete} disabled={loading}>
            {loading ? '削除中 ...' : '削除する'}
          </Button>
          <Button variant="outline" size="sm" onClick={() => { setShowConfirm(false); setB}}>
            キャンセル
          </Button>
        </div>
      </div>
    )
  }
}
```

```

    }

    // 初期表示: 削除ボタン
    return (
      <Button variant="ghost" size="sm" onClick={() => setShowConfirm(true)} aria-label="イベント削除" >
        <Trash2 className="h-4 w-4 mr-1" />
        削除
      </Button>
    )
  }
}

```

期待される動作:

初期状態:

[ 削除]

クリック後:

"このイベントを削除しますか？この操作は取り消せません。"

[削除する] [キャンセル]

「削除する」クリック後:

[削除中 ...] ← ボタン無効化

→ 成功: /events にリダイレクト

→ 失敗: エラーメッセージ表示

実装しないとどうなる？

- 確認ダイアログがないと、誤クリックで即座にイベントが削除されてしまいます
- `loading` 状態管理がないと、削除ボタンを連打できてしまい、APIに重複リクエストが送信されます

17.13 外部イベントのスクレイピングとインポート

このセクションで学ぶこと

- 外部サイトからのイベント情報スクレイピング
- 重複チェックのアルゴリズム（完全重複/類似イベント）
- 管理者専用機能の権限チェック

スクレイピング対象の定義

ファイル: `lib/scraping/bonsai-events.ts`

```
// lib/scraping/bonsai-events.ts

/** スクレイピング対象の地方とURL */
export const BONSAI_EVENT_SOURCES = [
  { region: '北海道', url: 'https://www.bonsai.co.jp/event/event_category/hokkaido/', prefecture: '北海道' },
  { region: '東北', url: 'https://www.bonsai.co.jp/event/event_category/tohoku/', prefecture: '東北' },
  { region: '関東', url: 'https://www.bonsai.co.jp/event/event_category/kanto/', prefecture: '関東' },
  { region: '信越', url: 'https://www.bonsai.co.jp/event/event_category/shinetsu/', prefecture: '信越' },
  { region: '北陸', url: 'https://www.bonsai.co.jp/event/event_category/hokuriku/', prefecture: '北陸' },
  { region: '東海', url: 'https://www.bonsai.co.jp/event/event_category/tokai/', prefecture: '東海' },
  { region: '近畿', url: 'https://www.bonsai.co.jp/event/event_category/kinki/', prefecture: '近畿' },
  { region: '中国', url: 'https://www.bonsai.co.jp/event/event_category/chugoku/', prefecture: '中国' },
  { region: '四国', url: 'https://www.bonsai.co.jp/event/event_category/shikoku/', prefecture: '四国' },
  { region: '九州', url: 'https://www.bonsai.co.jp/event/event_category/kyusyu/', prefecture: '九州' }
] as const

/** スクレイピングで取得したイベントの型 */
export interface ScrapedEvent {
  title: string
  startDate: Date | null
  endDate: Date | null
  prefecture: string | null
  city: string | null
  venue: string | null
  organizer: string | null
  admissionFee: string | null
  hasSales: boolean
  description: string
  externalUrl: string | null
  sourceRegion: string
  sourceUrl: string
}
```

インポート Server Actions

ファイル: `lib/actions/event-import.ts`

```

// lib/actions/event-import.ts
'use server'

import { prisma } from '@lib/db'
import { revalidatePath } from 'next/cache'
import { scrapeAllEvents, type ScrapedEvent } from '@lib/scraping/bonsai-events'
import { requireAdmin } from '@lib/actions/utils'

/** 重複タイプ */
export type DuplicateType = 'exact' | 'similar' | null
// exact: 完全重複（同タイトル・同期間・同都道府県）→ 自動除外
// similar: 類似イベント（類似タイトル・同都道府県・異なる期間）→ 黄色警告
// null: 重複なし

/**
 * タイトルの類似度を計算
 * - 完全一致
 * - 先頭10文字が一致
 * - 一方が他方を含む
 */
function isSimilarTitle(title1: string, title2: string): boolean {
  const normalized1 = title1.replace(/\s+/g, '').toLowerCase()
  const normalized2 = title2.replace(/\s+/g, '').toLowerCase()
  if (normalized1 === normalized2) return true
  if (normalized1.substring(0, 10) === normalized2.substring(0, 10)) return true
  if (normalized1.includes(normalized2) || normalized2.includes(normalized1)) return true
  return false
}

/**
 * 重複チェック: N+1クエリを回避するため、対象都道府県を一括取得してメモリ上で検索
 */
async function checkDuplicates(events: ScrapedEvent[]): Promise<Map<string, DuplicateCheck>> {
  const prefectures = [ ... new Set(events.map(e => e.prefecture).filter(Boolean)) ]
  const existingEvents = await prisma.event.findMany({
    where: {
      isHidden: false,
      ... (prefectures.length > 0 && { prefecture: { in: prefectures as string[] } }),
    },
    select: { title: true, startDate: true, endDate: true, prefecture: true },
  })

  // 都道府県別にグループ化
  const eventsByPrefecture = new Map<string, typeof existingEvents>()
  for (const existing of existingEvents) {
    const pref = existing.prefecture || ''
    if (!eventsByPrefecture.has(pref)) eventsByPrefecture.set(pref, [])
    eventsByPrefecture.get(pref)!.push(existing)
  }

  // 各イベントについて重複チェック

```

```

const duplicateMap = new Map<string, DuplicateCheckResult>()
for (const event of events) {
  const candidates = eventsByPrefecture.get(event.prefecture || '') || []
  // candidates内でタイトル類似 + 日付一致/不一致をチェック
  // ...
}
return duplicateMap
}

/** 全地方のイベントをスクレイピング（プレビュー用） */
export async function scrapeExternalEvents() {
  const { error } = await requireAdmin()
  if (error) return { error }

  const scrapedEvents = await scrapeAllEvents()
  const duplicateMap = await checkDuplicates(scrapedEvents)

  // 完全重複(exact)を除外し、類似(similar)は警告付きで返す
  let filteredCount = 0
  const events: ImportableEvent[] = []
  scrapedEvents.forEach((event, index) => {
    const duplicateResult = duplicateMap.get(event.title) || { type: null }
    if (duplicateResult.type === 'exact') {
      filteredCount++
      return
    }
    events.push(toImportableEvent(event, index, duplicateResult))
  })

  return { events, filteredCount }
}

/** 選択されたイベントをインポート */
export async function importSelectedEvents(events: ImportableEvent[]) {
  const { userId, error } = await requireAdmin()
  if (error) return { error }

  let importedCount = 0
  for (const event of events) {
    if (!event.startDate) continue

    // 念のため再度重複チェック
    const existing = await prisma.event.findFirst({
      where: { title: event.title, startDate: new Date(event.startDate), isHidden: false }
    })
    if (existing) continue

    await prisma.event.create({
      data: {
        title: event.title,
        startDate: new Date(event.startDate),
        endDate: event.endDate ? new Date(event.endDate) : null,
        prefecture: event.prefecture || null,
      }
    })
    importedCount++
  }
}

```

```

        // ... 他のフィールド
        createdBy: userId!,
    },
    })
    importedCount++
}

revalidatePath('/events')
return { success: true, importedCount }
}

```

期待される動作:

管理者がスクレイピング実行:

1. `scrapeExternalEvents()` を呼び出し
2. 外部サイトから30件のイベントを取得
3. 重複チェック: 5件が完全重複(除外)、2件が類似(警告付き)
4. 結果: { `events`: [25件], `filteredCount`: 5 }

管理者が10件を選択してインポート:

1. `importSelectedEvents(selectedEvents)` を呼び出し
2. 各イベントをDBに保存 (再度重複チェック付き)
3. 結果: { `success`: true, `importedCount`: 10 }

実装しないとどうなる?

- スクレイピング機能がないと、管理者が外部サイトのイベントを1件ずつ手入力する必要があります
- 重複チェックがないと、同じイベントが何度もインポートされてユーザーが混乱します
- `requireAdmin` チェックがないと、一般ユーザーが大量のイベントをインポートできてしまいます

17.14 Cronジョブ: 古いイベントの自動クリーンアップ

このセクションで学ぶこと

- Vercel Cron Jobsの設定と認証
- Route Handlerによる定期実行APIの実装
- データベースの自動メンテナンス

Cron設定

ファイル: `vercel.json`

```
{
  "crons": [
    {
      "path": "/api/cron/cleanup-events",
      "schedule": "0 0 1 * *"
    }
  ]
}
```

`"0 0 1 * *"` はcron式で「毎月1日の0時0分」を意味します。

分	時	日	月	曜日	
0	0	1	*	*	
				└	すべての曜日
			└		すべての月
		└			1日
	└				0時
└					0分

クリーンアップRoute Handler

ファイル: `app/api/cron/cleanup-events/route.ts`

```

// app/api/cron/cleanup-events/route.ts
import { NextRequest, NextResponse } from 'next/server'
import { prisma } from '@lib/db'
import { logger } from '@lib/logger'
import { EVENT_RETENTION_MONTHS } from '@lib/constants/limits'

/** Cron用のシークレットキーを検証 */
function validateCronSecret(request: NextRequest): boolean {
  const authHeader = request.headers.get('authorization')
  const cronSecret = process.env.CRON_SECRET
  if (!cronSecret) {
    return authHeader === `Bearer ${process.env.VERCEL_CRON_SECRET}`
  }
  return authHeader === `Bearer ${cronSecret}`
}

/**
 * GET /api/cron/cleanup-events
 * 終了日から6ヶ月以上経過したイベントを削除
 */
export async function GET(request: NextRequest) {
  // 認証チェック: 不正なアクセスを拒否
  if (!validateCronSecret(request)) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
  }

  try {
    // 6ヶ月前の日付を計算
    const sixMonthsAgo = new Date()
    sixMonthsAgo.setMonth(sixMonthsAgo.getMonth() - EVENT_RETENTION_MONTHS)

    // 終了日がnullの場合は開始日を使用して削除判定
    const result = await prisma.event.deleteMany({
      where: {
        OR: [
          {
            endDate: { not: null, lt: sixMonthsAgo },
          },
          {
            endDate: null,
            startDate: { lt: sixMonthsAgo },
          },
        ],
      },
    })

    logger.info(`[Cron] Deleted ${result.count} old events`)

    return NextResponse.json({
      success: true,
      deletedCount: result.count,
    })
  }
}

```

```

        cutoffDate: sixMonthsAgo.toISOString(),
      })
    } catch (error) {
      logger.error('[Cron] Event cleanup error:', error)
      return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
    }
  }
}

```

定数値:

- `EVENT_RETENTION_MONTHS = 6` -- イベント保持期間 (`lib/constants/limits.ts` で定義)

期待される動作:

毎月1日 0:00(UTC)に自動実行:

2026年8月1日のクリーンアップ:

基準日: 2026年2月1日 (6ヶ月前)

対象:

- endDateが2026年2月1日より前のイベント
- endDateがnullで、startDateが2026年2月1日より前のイベント

結果: { success: true, deletedCount: 15, cutoffDate: "2026-02-01T ..." }

セキュリティ:

- CronジョブはVercelが自動的にAuthorizationヘッダーを付与
- `CRON_SECRET` 環境変数で外部からの不正アクセスを防止
- 認証に失敗すると401エラーを返す

実装しないとどうなる?

- 古いイベントが永遠にデータベースに残り続け、ストレージコストが増大します
- イベント一覧の取得クエリが遅くなります (数万件のレコードを走査するため)
- 認証チェックがないと、誰でもこのAPIを叩いてイベントを削除できてしまいます

17.15 ローディングとエラーハンドリング

このセクションで学ぶこと

- `loading.tsx` によるスケルトンUI
- `error.tsx` によるエラーバウンダリ
- レイアウトシフト防止のテクニック

ローディングスケルトン

ファイル: `app/(main)/events/loading.tsx`

```
// app/(main)/events/loading.tsx
export default function EventsLoading() {
  return (
    <div className="space-y-6">
      {/* ヘッダースケルトン */}
      <div className="flex items-center justify-between">
        <div className="h-8 w-32 bg-muted rounded animate-pulse" />
        <div className="h-10 w-40 bg-muted rounded-lg animate-pulse" />
      </div>

      {/* 地域フィルタースケルトン */}
      <div className="bg-card rounded-lg border p-4 space-y-4">
        <div className="h-5 w-32 bg-muted rounded animate-pulse" />
        <div className="flex flex-wrap gap-2">
          {[ ... Array(8) ].map((_, i) => (
            <div key={i} className="h-8 w-20 bg-muted rounded-full animate-pulse" />
          ))}
        </div>
      </div>

      {/* カレンダースケルトン */}
      <div className="bg-card rounded-lg border">
        <div className="p-4 border-b flex items-center justify-between">
          <div className="h-8 w-8 bg-muted rounded animate-pulse" />
          <div className="h-6 w-32 bg-muted rounded animate-pulse" />
          <div className="h-8 w-8 bg-muted rounded animate-pulse" />
        </div>
        <div className="grid grid-cols-7">
          {[ ... Array(35) ].map((_, i) => (
            <div key={i} className="h-24 border-b border-r bg-muted/10 animate-pulse" />
          ))}
        </div>
      </div>
    </div>
  )
}
```

スケルトンUIのポイント:

- 実際のレイアウトと同じ構造を持つことで、レイアウトシフト（CLS）を防止
- `animate-pulse` クラスでパルスアニメーションを適用
- 実データが読み込まれると、スケルトンが自動的に実コンテンツに置き換わる

エラーバウンダリ

ファイル: `app/(main)/events/error.tsx`

```
// app/(main)/events/error.tsx
'use client' // error.tsxは必ずClient Component

import { PageError } from '@components/common/PageError'

export default function EventsError({
  error,
  reset,
}: {
  error: Error & { digest?: string }
  reset: () => void
}) {
  return (
    <PageError
      error={error}
      reset={reset}
      title="イベント一覧を読み込みません"
      description="イベントの取得に失敗しました。再試行してください。"
    />
  )
}
```

`reset` 関数の仕組み:

- Next.jsが自動的に提供する関数で、エラーバウンダリをリセットしてページの再レンダリングを試みる
- 一時的なエラー（ネットワーク切断など）の場合、`reset()` で復帰できる

実装しないとどうなる？

- `loading.tsx` がないと、データ取得中に白い画面が表示され、ユーザーは「壊れた？」と思います
- `error.tsx` がないと、エラー発生時にアプリ全体がクラッシュする可能性があります
- スケルトンが実レイアウトと合っていないと、読み込み後にガタつきが発生します

17.16 定数値の管理

このセクションで学ぶこと

- アプリケーション全体の制限値を一元管理する方法
- マジックナンバーを避ける理由

定数定義

**ファイル: `lib/constants/limits.ts` ** (イベント関連の抜粋)

```
// lib/constants/limits.ts

/** カレンダーの1日に表示するイベント数の上限 */
export const CALENDAR_EVENTS_PER_DAY = 3

/** イベントタイトルの最大文字数 */
export const MAX_EVENT_TITLE_LENGTH = 100

/** サイドバーに表示する直近イベント数 */
export const UPCOMING_EVENTS_LIMIT = 10

/** イベント保持期間（月数） - これを超えたイベントはCronで削除 */
export const EVENT_RETENTION_MONTHS = 6
```

なぜ定数ファイルで管理するのか？

例えば「カレンダーの1日に表示するイベント数」を変更したい場合、定数ファイルの1箇所を変更するだけで、カレンダーコンポーネントとテストの両方に反映されます。

```
// ❌ 悪い例: マジックナンバー
dayEvents.slice(0, 3) // 「3」って何?なぜ3?

// ✅ 良い例: 名前付き定数
dayEvents.slice(0, CALENDAR_EVENTS_PER_DAY) // 1日あたりの表示上限
```

演習問題

演習1: イベント検索機能の追加（難易度: 中）

イベントのタイトルでキーワード検索する機能を追加してください。

ヒント:

- URLパラメータ `q` で検索キーワードを受け取る
- Prismaの `contains` オプションを使用
- 検索入力フィールドをRegionFilterの上に配置

```
// getEvents関数に追加するWHERE条件
where: {
  title: { contains: keyword, mode: 'insensitive' },
}
```

演習2: 「今週のイベント」セクション（難易度: 低）

イベント一覧ページの上部に、今週開催されるイベントだけを表示するセクションを追加してください。

ヒント:

- `date-fns` の `startOfWeek` と `endOfWeek` を使用
- 今週のイベントを取得するServer Actionを追加
- カレンダーの上にハイライトセクションとして配置

演習3: イベントのCSVエクスポート（難易度: 高）

管理者がイベント一覧をCSVファイルとしてダウンロードできる機能を追加してください。

ヒント:

- Route Handler (`app/api/events/export/route.ts`) を作成
- `Content-Type: text/csv` でレスポンスを返す
- 管理者権限チェックを忘れずに

まとめ

本章で学んだ内容を振り返ります。

習得した技術

技術	使用箇所
Prismaモデル定義	<code>prisma/schema.prisma</code> -- Eventモデルのスキーマ定義
Server Actions	<code>lib/actions/event.ts</code> -- CRUD操作の実装
URLパラメータ連携	<code>RegionFilter</code> , <code>ShowPastToggle</code> -- フィルタ状態のURL管理
動的インポート	<code>EventCalendarWrapper</code> -- SSR無効化ラッパー
date-fns	<code>EventCalendar</code> , <code>EventCard</code> -- 日付操作とフォーマット
localStorage	<code>EventFilterPersistence</code> -- フィルタ設定の永続化
Cronジョブ	<code>app/api/cron/cleanup-events</code> -- 自動クリーンアップ
スクレイピング	<code>lib/scraping/bonsai-events.ts</code> -- 外部イベント取得
SEO対策	イベント詳細 -- generateMetadata, JSON-LD
エラーハンドリング	<code>error.tsx</code> , <code>loading.tsx</code> -- ユーザー体験の向上

関連する主要ファイル一覧

ファイルパス	役割
<code>prisma/schema.prisma</code>	Eventモデルの定義
<code>lib/prefectures.ts</code>	地方ブロック・都道府県の定数とユーティリティ
<code>lib/actions/event.ts</code>	イベントCRUDのServer Actions
<code>lib/actions/event-import.ts</code>	スクレイピング・インポートのServer Actions
<code>lib/scraping/bonsai-events.ts</code>	外部サイトスクレイピングロジック
<code>lib/constants/limits.ts</code>	制限値定数（表示件数、保持期間等）
<code>app/(main)/events/page.tsx</code>	イベント一覧ページ
<code>app/(main)/events/[id]/page.tsx</code>	イベント詳細ページ
<code>app/(main)/events/new/page.tsx</code>	新規登録ページ
<code>app/(main)/events/[id]/edit/page.tsx</code>	編集ページ
<code>app/(main)/events/loading.tsx</code>	ローディングスケルトン
<code>app/(main)/events/error.tsx</code>	エラーバウンダリ
<code>app/api/cron/cleanup-events/route.ts</code>	Cronジョブ（自動クリーンアップ）
<code>components/event/EventCard.tsx</code>	イベントカードコンポーネント
<code>components/event/EventList.tsx</code>	イベント一覧グリッド
<code>components/event/EventCalendar.tsx</code>	カレンダー本体
<code>components/event/EventCalendarWrapper.tsx</code>	カレンダーSSR無効化ラッパー
<code>components/event/EventForm.tsx</code>	作成・編集フォーム
<code>components/event/RegionFilter.tsx</code>	地域フィルター
<code>components/event/ShowPastToggle.tsx</code>	過去イベント表示切替
<code>components/event/EventFilterPersistence.tsx</code>	フィルター永続化
<code>components/event/DeleteEventButton.tsx</code>	削除ボタン
<code>components/event/EventActionsDropdown.tsx</code>	アクションドロップダウン

`vercel.json`

Cronジョブのスケジュール設定

次の章への橋渡し

第18章では、盆栽園マップ機能を実装します。地図ライブラリ（Leaflet）の導入、位置情報を使ったデータ表示、レビュー機能など、地理情報を扱うテクニックを学びます。